

● Maven Lifecycle

1. **Validate:** Ensures the project structure and configuration are correct before proceeding. The code and project setup are validated.
2. **Compile:** The source code is compiled into bytecode, generating .class files.
3. **Test:** Unit tests are run using testing frameworks like JUnit to ensure the correctness of the code.
4. **Package:** The compiled code is bundled into a distributable format like a .jar or .war, creating the artifact.
5. **Verify:** Integration tests are executed to verify the integrity and interoperability of the code with other components.
6. **Install:** The packaged artifact is installed in the local Maven repository (typically in the ~/.m2/repository folder), making it available for other projects as a dependency.
7. **Clean:** This phase removes artifacts from previous builds, ensuring a clean environment for new builds.
8. **Deploy:** The final built artifact is uploaded to a remote repository (like Nexus or Artifactory) or deployed to a production environment.

● Handling Dependency Issues and Installing Custom Java.

If you face dependency issues or need to install a specific Java version, follow these steps:

1. Install Java:

- Download the required Java version from the Oracle website.
- Copy the download link for the Java version suitable for your Linux system.

2. Transfer to the Linux Server:

- Use SCP (Secure Copy Protocol) or a tool like WinSCP to transfer the Java package from your Windows machine to your Linux server.
- Example SCP command:

```
scp -i <path_to_key> <source_file_path> ec2-user@<public_ip>:/tmp/
```

3. Extract and Install Java:

- SSH into your Linux server and go to the /tmp directory where you uploaded the Java file.

- Extract the file:

```
tar -xvf jdk-<version>-linux-x64.tar.gz
```

- Set the JAVA_HOME and PATH environment variables:

```
export JAVA_HOME=/path_to_extracted_java  
export PATH=$JAVA_HOME/bin:$PATH
```

- Apply the changes:

```
source ~/.bashrc
```

- Verify the Java installation:

```
java --version
```

4. Run Maven Commands:

- Now that Java is set up, you can run Maven commands like:

```
mvn clean install
```

Automating with Jenkins.

To automate Maven builds and deployments with Jenkins:

1. Set Up Jenkins:

- Go to Manage Jenkins → Global Tool Configuration.
- In the JDK section, specify the path to your installed Java by defining JAVA_HOME.

2. Install Deployment Plugins:

- In Jenkins, install plugins like "Deploy to Container" for Tomcat.
- After installation, configure your Jenkins job to add a post-build action to deploy your .war or .ear file.

3. Configure Job:

- In the job configuration, ensure Maven builds are included (using mvn clean install in the build steps).

- Configure the "Deploy war/ear to a container" step by providing the necessary details for your Tomcat server.

4. Verify Deployment:

Once the job completes successfully, access the application via the server's public IP in a web browser to ensure the deployment was successful.